CO 3 AT 2

CASE STUDY

NAME: **S.SRI MATHI**

REG.NO: **192424095**

COURSE CODE: **CSA0603**

COURSE NAME: **DESIGN AND ANALYSIS OF ALGORITHM**

DATE: 05.08.2026

1.INTRODUCTION

Financial institutions such as banks, payment gateways, insurance companies, and stock markets process thousands of transactions every second. These transactions include deposits, withdrawals, online payments, fund transfers, and purchases. Before generating reports, calculating balances, detecting fraud, or performing financial analysis, transaction records must be sorted efficiently.

If sorting takes too much time or memory, report generation becomes slower, affecting decision-making and customer service. Therefore, selecting an efficient sorting algorithm is important for improving the overall performance of financial data processing systems.

2. PROBLEM STATEMENT

Develop a financial transaction processing system that sorts transaction records based on transaction amount. Implement both Merge Sort and Quick Sort using the Divide and Conquer technique. Compare the algorithms in terms of execution speed, stability, memory usage, and time complexity. Based on the comparison, determine the most suitable algorithm for processing financial transaction data.

3. ALGORITHM USED

This case study uses the **Divide and Conquer** strategy.

Merge Sort

Merge Sort divides the transaction list into smaller sublists until each sublist contains only one element. It then merges the sublists in sorted order to produce the final sorted transaction list.

Quick Sort

Quick Sort selects a pivot element and partitions the transactions into two groups: elements smaller than the pivot and elements larger than the pivot. The same process is recursively applied until the entire list is sorted.

4. JUSTIFICATION FOR CHOOSING THE ALGORITHM

Merge Sort and Quick Sort are selected because they are among the most efficient comparison-based sorting algorithms for large datasets.

Merge Sort guarantees **O(n log n)** time complexity in all cases and preserves the relative order of equal elements, making it a stable sorting algorithm. This property is useful in financial applications where multiple transactions may have the same amount but should remain in chronological order.

Quick Sort is widely used because of its excellent average-case performance and low memory consumption. It performs sorting in-place and is generally faster than Merge Sort for most practical applications when an appropriate pivot is selected.

Both algorithms apply the Divide and Conquer technique, making them suitable for processing large volumes of financial transaction records efficiently.

5. COMPLEXITY ANALYSIS

Analysis

- Merge Sort provides consistent performance regardless of input order.
- Quick Sort performs faster in most average situations but can degrade to **$O(n^2)$** when poor pivot selection occurs.
- Merge Sort requires additional memory for merging.

- Quick Sort requires very little extra memory because sorting is performed in-place.

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)

6. MANUAL EXECUTION (STEP-BY-STEP PROCESS)

Sample Transaction Amounts

```
4500
1200
9800
3500
6700
1500
```

Merge Sort

Step 1

```
4500 1200 9800 3500 6700 1500
```

Split into two halves

```
4500 1200 9800

3500 6700 1500
```

Step 2

Further divide

```
4500

1200 9800

3500

6700 1500
```

Step 3

Divide again

```
4500

1200

9800

3500

6700

1500
```

Step 4

Merge in sorted order

```
1200 9800

1500 6700
```

Step 5

Merge larger groups

```
1200 4500 9800
```

```
1500 3500 6700
```

Step 6

Final Merge

```
1200 1500 3500 4500 6700 9800
```

Quick Sort

Initial List

```
4500 1200 9800 3500 6700 1500
```

Choose Pivot

```
4500
```

Partition

Left

```
1200 3500 1500
```

Pivot

```
4500
```

Right

```
9800 6700
```

Sort Left

```
1200 1500 3500
```

Sort Right

```
6700 9800
```

Final Result

```
1200 1500 3500 4500 6700 9800
```

7. APPLICATIONS

Merge Sort and Quick Sort are widely used in real-world financial systems.

- Banking transaction processing
- Online payment systems
- Credit and debit card transaction analysis
- Stock market trading platforms
- Financial report generation
- Tax and audit record processing
- Insurance claim processing
- Digital wallet transaction management
- Fraud detection systems
- Financial data analytics

8. ADVANTAGES

Merge Sort	Quick Sort
Guaranteed O(n log n) performance.	Excellent average-case performance.
Stable sorting algorithm.	Faster than many sorting algorithms in practice.
Suitable for large datasets.	Requires very little additional memory.
Efficient for linked lists.	Performs sorting in-place.
Ideal for external sorting where data is stored on disk.	Efficient for large datasets stored in memory.

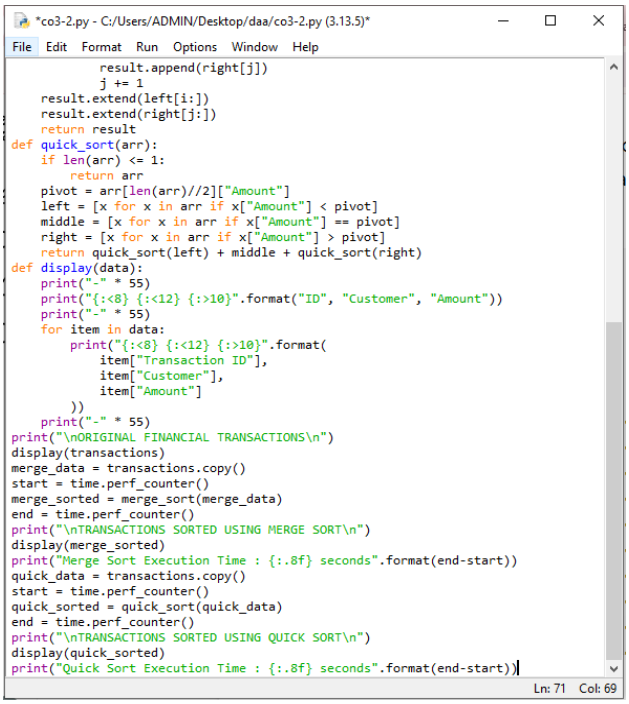
9. LIMITATIONS

Merge Sort	Quick Sort
Requires additional memory during merging.	Worst-case complexity is $O(n^2)$.
Slightly slower than Quick Sort for smaller datasets.	Performance depends on pivot selection.
More memory-intensive.	Not a stable sorting algorithm.
Efficient for linked lists.	Performs sorting in-place.
Ideal for external sorting where data is stored on disk.	Efficient for large datasets stored in memory.

10. Comparison with Other Algorithms

Feature	Bubble Sort	Selection Sort	Insertion Sort	Merge Sort	Quick Sort
Technique	Comparison	Comparison	Comparison	Divide & Conquer	Divide & Conquer
Best Case	$O(n)$	$O(n^2)$	$O(n)$	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n^2)$
Stable	Yes	No	Yes	Yes	No
Extra Memory	No	No	No	Yes	No
Suitable for Large Data	No	No	No	Yes	Yes

11. SCREENSHOT OF THE PROGRAM



```
*co3-2.py - C:/Users/ADMIN/Desktop/daa/co3-2.py (3.13.5)*
File Edit Format Run Options Window Help
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]["Amount"]
    left = [x for x in arr if x["Amount"] < pivot]
    middle = [x for x in arr if x["Amount"] == pivot]
    right = [x for x in arr if x["Amount"] > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def display(data):
    print("-" * 55)
    print("{:<8} {:<12} {:>10}".format("ID", "Customer", "Amount"))
    print("-" * 55)
    for item in data:
        print("{:<8} {:<12} {:>10}".format(
            item["Transaction ID"],
            item["Customer"],
            item["Amount"]
        ))
    print("-" * 55)
print("\nORIGINAL FINANCIAL TRANSACTIONS\n")
display(transactions)
merge_data = transactions.copy()
start = time.perf_counter()
merge_sorted = merge_sort(merge_data)
end = time.perf_counter()
print("\nTRANSACTIONS SORTED USING MERGE SORT\n")
display(merge_sorted)
print("Merge Sort Execution Time : {:.8f} seconds".format(end-start))
quick_data = transactions.copy()
start = time.perf_counter()
quick_sorted = quick_sort(quick_data)
end = time.perf_counter()
print("\nTRANSACTIONS SORTED USING QUICK SORT\n")
display(quick_sorted)
print("Quick Sort Execution Time : {:.8f} seconds".format(end-start))
Ln: 71  Col: 69
```

```
*co3-2.py - C:/Users/ADMIN/Desktop/daa/co3-2.py (3.13.5)*
File Edit Format Run Options Window Help
import time
transactions = [
    {"Transaction ID": "T101", "Customer": "Arun", "Amount": 4500},
    {"Transaction ID": "T102", "Customer": "Priya", "Amount": 1200},
    {"Transaction ID": "T103", "Customer": "Rahul", "Amount": 9800},
    {"Transaction ID": "T104", "Customer": "Divya", "Amount": 3500},
    {"Transaction ID": "T105", "Customer": "Karthik", "Amount": 6700},
    {"Transaction ID": "T106", "Customer": "Meena", "Amount": 1500},
    {"Transaction ID": "T107", "Customer": "Ajay", "Amount": 8900},
    {"Transaction ID": "T108", "Customer": "Nisha", "Amount": 2500},
    {"Transaction ID": "T109", "Customer": "Ravi", "Amount": 5200},
    {"Transaction ID": "T110", "Customer": "Anu", "Amount": 7200}
]
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i]["Amount"] <= right[j]["Amount"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]["Amount"]
Ln: 71  Col: 69
```

12. OUTPUT SCREENSHOT

A) Original transaction data

```
ORIGINAL FINANCIAL TRANSACTIONS

-------------------------------------------------------
ID       Customer         Amount
-------------------------------------------------------
T101     Arun               4500
T102     Priya              1200
T103     Rahul              9800
T104     Divya              3500
T105     Karthik            6700
T106     Meena              1500
T107     Ajay               8900
T108     Nisha              2500
T109     Ravi               5200
T110     Anu                7200
-------------------------------------------------------
```

B) Sorted output using Merge Sort

```
TRANSACTIONS SORTED USING MERGE SORT

-----------------------------------------------------
ID       Customer         Amount
-----------------------------------------------------
T102     Priya              1200
T106     Meena              1500
T108     Nisha              2500
T104     Divya              3500
T101     Arun               4500
T109     Ravi               5200
T105     Karthik            6700
T110     Anu                7200
T107     Ajay               8900
T103     Rahul              9800
-----------------------------------------------------
Merge Sort Execution Time : 0.00008890 seconds
```

C) Sorted output using Quick Sort

```
TRANSACTIONS SORTED USING QUICK SORT

-----------------------------------------------------
ID       Customer         Amount
-----------------------------------------------------
T102     Priya              1200
T106     Meena              1500
T108     Nisha              2500
T104     Divya              3500
T101     Arun               4500
T109     Ravi               5200
T105     Karthik            6700
T110     Anu                7200
T107     Ajay               8900
T103     Rahul              9800
-----------------------------------------------------
Quick Sort Execution Time : 0.00013460 seconds
```

13. CONCLUSION

Financial systems require efficient sorting algorithms to process large numbers of transaction records quickly and accurately. Merge Sort provides guaranteed **O(n log n)** performance and maintains stability, making it suitable when preserving the order of equal transactions is important. Quick Sort offers superior average-case performance with lower memory usage, making it ideal for high-speed in-memory processing.

For financial systems where transaction order must be preserved, **Merge Sort is the preferred choice** because of its stability and consistent performance. For applications that prioritize execution speed and memory efficiency, **Quick Sort is more suitable** when an effective pivot selection strategy is used.